

trade-ledger

User guide · version 0.2.0 · 10 September 2026

A local trade and investment logger with one tab per venue account, a journal measured in R, analytics, portfolio, German tax record keeping, a weekly PDF report, and a Bot Center that monitors, configures and controls trading bots.

Everything runs on your own machine. There is no account, no cloud, no telemetry. The database is a single file under `data/`.

The five rules the app enforces

- Read-only venue keys. The app only ever reads from Trading 212, OKX and Kraken. It has no code path that places, cancels or withdraws.
- A trade is written down before the order. The Journal refuses a plan without entry, stop, risk and a written reason.
- Paper, demo and live never mix. Every statistic is filtered by mode; the mode is shown in the title.
- Results are in R. $R = \text{result} \div \text{planned risk}$. Euros are shown second.
- Tax figures are calculations, not advice. Every Steuer screen says Berechnung — mit Steuerberater prüfen.

Contents

1. Install and first start
2. Accounts and venue keys
3. Getting data in
4. The Journal
5. Analytics and the stage gate
6. Portfolio and prices
7. Steuer
8. Reports
9. Bot Center
10. Writing a new bot
11. Command line reference
12. A weekly routine
13. Troubleshooting

1. Install and first start

Requirements: Python 3.13 or newer with uv, Node 22 or newer. Tested on macOS.

```
git clone https://github.com/AmirrezaRoodsaz/trade-ledger.git
cd trade-ledger
uv sync                                # backend dependencies
cd frontend && npm ci && npm run build  # builds the web UI into frontend/dist
cd ..
cp .env.example .env                  # optional: read-only API keys go here
uv run trade-ledger serve              # opens http://127.0.0.1:8642
```

On the first start the app creates `data/ledger.db`. Without a `.env` everything still works: you import CSV files and enter trades by hand.

Secrets stay local. `.env`, `data/`, exports, screenshots and notes are gitignored. A secret scanner runs before every commit and in CI. Never paste a key anywhere else.

The left navigation

Entry	What it holds
Dashboard	Paper and live stat cards, open trades, last sync per account, tax meters, portfolio value, bots
One entry per account	Overview, Transactions, Trades, Holdings for that venue account (badge: live / paper / demo)
Journal	Plan, open, close and review trades; monthly calendar; daily note
Analytics	Expectancy, profit factor, drawdown, breakdowns, equity curve, stage gate
Portfolio	Holdings, value curve, allocation, dividends, fees, returns
Steuer	FIFO lots, § 23 / § 20 pots, Freigrenze meters, Anlage suggestions, exports
Reports	Weekly PDF
Bots	Fleet, bot detail, presets, strategy dashboard
Settings	Accounts, key presence, sync, CSV import, playbooks, tags, Telegram, criteria

2. Accounts and venue keys

Settings → Accounts → New account. Give it a name, the venue, the kind (broker invest, crypto spot, crypto derivatives, wallet), the mode (live, paper, demo) and the credential prefix that names its keys in `.env`. The tax wallet defaults to the account name; FIFO is computed per tax wallet.

Each account becomes a tab in the left navigation. Create separate accounts for live and practice.

Creating read-only keys

Venue	Where	Keys in <code>.env</code>	Covers	Limits
Trading 212	Mobile app → Settings → API (Beta). Switch to Practice first for a demo key.	T212_LIVE_API_KEY / _SECRET, T212_DEMO_...	Fills, dividends, cash transactions, positions	Invest and ISA only. CFD history via CSV. Withholding tax entered by hand.
OKX	Web → Profile → API → Create V5 key, permission Read only. Remember the passphrase.	OKX_LIVE_API_KEY / _SECRET / _PASSPHRASE, OKX_DEMO_...	Spot trades, deposits, withdrawals, staking, balances	About three months of history; import the quarterly archive CSV first.
Kraken	Web → Settings → API → Add key, Query permissions only.	KRAKEN_LIVE_API_KEY / _SECRET	Trades, deposits, withdrawals, staking, balances	No sandbox: a paper or demo Kraken account still reads the live API.

After editing `.env`, restart the server. Settings → Environment shows a green or red dot per key; values are never shown.

Grant read permissions only for the app's keys. Bot keys (section 9) are the only trade-capable keys, and they live in a separate file that only the bot process opens.

3. Getting data in

Sync (API)

On an account tab or in Settings, press Sync. The app pulls everything since the last successful run minus seven days, deduplicates on the venue's own ids, and records a sync run with added and skipped counts and any warning. Rows in a non-EUR currency are marked pending and valued as soon as ECB rates are fetched.

CSV import

Settings → Import CSV: choose the account, the format (Trading 212, OKX, Kraken ledger, Binance transaction history, or the generic format), and the file. A preview lists every parsed row and every row that failed with the reason; commit only writes the good rows. Importing the same file twice adds nothing.

Generic CSV columns: `date`, `type`, `symbol`, `asset_class`, `quantity`, `price`, `currency`, `fee`, `fee_currency`, `amount_eur`, `external_id`, `note`.

Manual transactions

Account tab → Transactions → Add. Quantity, amounts and fees are always positive; the type carries the direction (buy, sell, deposit, withdrawal, transfer in/out, dividend, interest, fee, staking reward, airdrop, split).

Prices

Crypto is priced from Bitstamp, stocks and ETFs from Stooq, currencies from the ECB. Press Refresh prices on the Portfolio page after your first import; the same button resolves pending EUR values.

4. The Journal

The Journal is the heart of the app. A trade is one idea with a plan, fills, a result and a review.

Step	What you do	What the app checks
Plan	Account, instrument, direction, entry, stop, target, risk in €, why this trade, where it is wrong, what would make you skip it	Refuses without entry, stop, risk and the reason. Suggests the quantity: $\text{risk} \div \text{entry} - \text{stop} $.
Open	Link the real fills from the account, or enter a manual fill (paper)	Only from Planned; recomputes average entry, quantity, fees
Close	Link exit fills or enter one	Computes result after fees and $R = \text{result} \div \text{risk}$
Review	Adherence yes/no, mistake tag, emotion, one-line lesson	A losing trade that followed the rules is a good trade; a winner that broke them gets adherence = no

Mistake tags: moved_stop, no_stop, oversized, revenge, fomo, early_exit, late_entry, off_plan.

Trade detail

- Price chart with entry, stop, target and exit markers (daily candles).
- MAE and MFE: how far the trade went against you and for you, in € and R. Press Compute on the trade or recompute all closed trades from Analytics.
- Screenshots: upload PNG, JPG or WEBP; they are stored locally and shown as thumbnails.

Calendar and daily note

The monthly calendar shows trades and R per day, coloured by sign. The daily note (text, mood, hours) lives next to it.

Playbooks

Settings → Playbooks. A playbook is a strategy with versioned rules text. Every plan links to a playbook version, so a rule change is a new version, never an edit. Analytics can break results down by playbook.

Notes export for Obsidian

Settings → Notes export or `uv run trade-ledger export-notes` writes one markdown note per trade in the trade-template frontmatter format to the configured folder, so an external stats script keeps working.

5. Analytics and the stage gate

The filter bar (accounts, mode, playbook, instrument, tag, dates) drives every panel. Mode defaults to paper; choose all explicitly to mix.

Metric	Meaning
Expectancy R	Mean R per closed trade
Profit factor	Sum of winning R ÷ sum of losing R
Win rate, payoff	Share of winners; average win ÷ average loss
SQN	Expectancy ÷ standard deviation of R × $\sqrt{N}$ (Van Tharp)
Max drawdown	Largest peak-to-trough drop of cumulative R and of cumulative €
Adherence	Share of closed trades that followed the rules
Cost of mistakes	R and € lost on trades with a mistake tag
MAE / MFE, capture	Average adverse and favourable excursion; result ÷ MFE

Charts: equity curve with drawdown, R histogram, breakdown table (playbook, instrument, weekday, month, hold time, tag, emotion, account), MAE vs MFE scatter.

Stage gate

The panel checks the same numbers as the money-side script: 30 closed paper trades or 50 live, expectancy above zero, adherence at least 90 %, drawdown at most 20 % of the stage capital. Stage capital is set in Settings (paper 2.500 €, live 250 € by default). Each check shows pass or fail with the reason.

6. Portfolio and prices

- Holdings per account and combined: quantity, average cost, value, unrealised P&L, weight, and for crypto the days until the 12-month holding period ("steuerfrei" once passed).
- Value chart over a date range; days without a price are listed.
- Allocation pies by asset class, venue and instrument.
- Cash flows: deposits, withdrawals, dividends, interest, fees.
- Dividends per year, by instrument and month, with a naive projection for the next 12 months.
- Returns: time-weighted (TTWROR) and money-weighted (XIRR), start and end value, net flows.

Missing prices show as "—", never as zero. Press Refresh prices to fetch.

7. Steuer

Pick the year and the mode (live by default). Every response carries the disclaimer and the form status
Formstand: VZ 2025, geprüft 2026-09-06. Line numbers come from secondary sources and must be verified.

Tab	What it shows
Meters	§ 23 Freigrenze 1.000 € (all-or-nothing warning when exceeded), § 22 Nr. 3 Freigrenze 256 €, Sparerpauschbetrag 1.000 €
§ 23	FIFO disposals per wallet with holding days, tax-free flag, gain; open lots with the 12-month countdown
§ 20	Share gains and losses, Termingeschäfte, dividends, interest, foreign withholding tax
KAP-INV	Per fund: type, Teilfreistellung rate (shown, not applied), distributions, Vorabpauschale, sale gains and losses
Jahresendbestände	Holdings per wallet at 31.12 in EUR
Venues	Per account: gross proceeds, acquisitions, count, deposits, withdrawals, for reconciling with DAC8 reports
Anlage	Suggested figures for Anlage SO, KAP and KAP-INV lines, labelled "Vorschlag, nicht amtlich"
Warnings	Unlinked transfers, missing prices, disposals without a regime

Exports: disposal list, lots, Blockpit CSV, CoinTracking CSV. Command line: `uv run trade-ledger tax-year 2025`.

Transfers between your own wallets must be linked (same link id on the outgoing and incoming rows) so the lots keep their acquisition date. Unlinked transfers appear in Warnings.

8. Reports

Reports → Generate (week end date, mode) builds a one-page PDF: headline stats for the week and all-time, equity curve, monthly calendar, open trades, open crypto lots with the countdown, tax meters. Files are listed with download links. Command line: `uv run trade-ledger report --week 2026-09-13 --mode paper`.

9. Bot Center

Bots are separate Python processes. The app is their registry, health monitor, configuration source, command queue, alert channel and, through the Journal, their trade record. The app never holds trade keys and never places orders.

Register a bot

- Bots → New bot: name, account, strategy, host (local if the app's supervisor should launch it, remote for a VPS), schedule (every 1 h / 4 h / 24 h at HH:MM UTC), grace minutes, preset version, optional stage capital.
- The bearer token is shown once. Copy it. Rotate it from the card menu if it leaks.
- Assign a preset version (Bots → Presets). Without one the bot finishes every run as skipped.

Write the env file

```
mkdir -p data/bots/<slug>
cp bots/template/bot.env.example data/bots/<slug>/.env
chmod 600 data/bots/<slug>/.env
# fill in BOT_TOKEN and the exchange keys (read + trade, no withdrawal, IP-restricted)
```

Give each bot its own venue sub-account: on spot the bot reads its position from the coin balance, so a coin you bought by hand in the same account looks like a position it opened.

Run it

```
uv run trade-bot --bot <slug> --dry-run      # everything except the order
EXCHANGE_ID=fake uv run trade-bot --bot <slug> --dry-run  # no keys: smoke-test the loop
uv run trade-bot --bot <slug>                # real run (demo account first)
```

A local bot is then launched on schedule by the supervisor while `trade-ledger serve` runs. A remote bot is scheduled by the shipped `systemd` unit and timer and reaches the app over the network; the timer cadence must match the bot's schedule, otherwise the heartbeat rule fires.

What a run does

Fetch config → heartbeat → reconcile the journal against the exchange (mismatch: halt or flat, per preset) → apply pending commands → candles → strategy → exits before entries → size from risk % and stop distance → plan the trade in the Journal → order with the trade ref as client-order id → resting stop → open the trade with the fill → push state.

If the app is unreachable, the bot places no order. A stop that the venue refuses makes the bot close the position and cancel the plan. An order is never re-sent.

The fleet page

One card per bot: status light (ok, running, paused, error, stale, disabled), venue and mode, host, last heartbeat and next run, equity and drawdown, K1–K5 chips, positions with a red marker when one has no stop, readiness ring, Pause and Run now. Cards refresh every 30 seconds.

Bot detail tabs

Tab	Contents
Overview	State, kill rules with values and thresholds, positions with stop flags, reconciliation banner, key presence
Runs	Every run with status and summary; click for the log
Timeline	Events (heartbeat, info, warning, error, kill rule, command, config applied, reconcile, order)

Config	Assigned preset version, hint when a newer version exists, apply with a typed reason (required for live)
Controls	Pause entries · Resume (reason) · Emergency flat (type the bot name) · Run now · Dry-run on/off; pending and acknowledged commands
Alerts	This bot's alerts with acknowledge buttons
Strategy	Backtest, incubation and real-money results side by side; equity overlay; current stage gate; drill checklist; readiness; backtest upload

Controls are soft

A command is queued in the app and executed by the bot on its next check; the UI shows pending until the bot acknowledges it. Emergency flat is executed by the bot: cancel every order, close every position, then pause.

Kill rules

Rule	Trigger	Action	Evaluated by
K1 capital brake	Equity $\leq$ 80 % of stage capital or drawdown $\geq$ 20 %	Pause entries + critical alert	App, from the bot's state
K2 rolling edge	Profit factor $<$ 1,0 over the last 20 closed trades	Warning alert	App, from the journal
K3 streak	8 consecutive losses	Warning alert	App, from the journal
K4 integrity	Reconciliation mismatch, position without a stop, two missed runs	Critical alert; flat queued for a stop-less position	Bot reconciles, app judges
K5 heartbeat	No heartbeat within schedule + grace	Critical alert, status stale	App, independently

K1 needs an equity in EUR. On USDT-quoted markets the bot cannot report one, and K1 shows a warning instead of pretending to be fine. Alerts of the same kind repeat at most every six hours. A disabled bot is never alerted on.

Telegram

Set TELEGRAM_BOT_TOKEN and TELEGRAM_CHAT_ID in .env, restart, press Send test message in Settings. You get warnings and critical alerts, a daily summary at 07:00, and you can send /status, /pause slug, /resume slug reason, /flat slug CONFIRM, /runnow slug from that chat only.

Presets

A preset is a named, versioned parameter set: strategy params (Donchian: entry 55, exit 20, ATR length 20, ATR multiple 2), timeframe, pairs, risk %, max position %, leverage cap. Versions are immutable; edit means a new version. A bot pulls its assigned version at every run start.

Readiness and the strategy dashboard

Readiness 0–100 % has four stages with editable weights (Settings → Bot readiness & backtest criteria): backtest passed 25 %, five drills 15 % (capability test, dry run, self-check, recovery drill, scheduling), incubation 30 % (closed demo trades $\div$ 30), live 30 % (closed live trades $\div$ 50). A failing stage gate halves its stage.

Backtests are not run by the app. Upload a JSON result (period, data source, trades, expectancy R, profit factor, win rate, max drawdown, CAGR, benchmark) on the Strategy tab; it is judged against the criteria in Settings and shown next to the incubation and real-money columns.

10. Writing a new bot

Hand `bots/BOT_CONTRACT.md` to whoever writes the bot, Claude included. It is self-contained: the loop in order, every endpoint with its JSON, the strategy function signature, the environment variables, the kill rules a bot honours locally, and the acceptance tests it must pass.

```
uv run trade-bot new my-strategy
# creates bots/strategies/my-strategy.py (from the template) and data/bots/my-strategy/.env
# then: implement signals(candles, params, open_positions), register it in STRATEGIES,
#       create the bot in the UI, paste the token into the env file
```

The strategy function is pure: the same candles always give the same signals. Everything else (journal, orders, stops, reconciliation, commands) is done by the shared runner.

11. Command line reference

```
uv run trade-ledger serve [--port N] [--no-browser]
uv run trade-ledger sync (--account NAME | --all)
uv run trade-ledger import --account NAME --format trading212|okx|kraken|binance|generic FILE
uv run trade-ledger export-notes [--dir DIR]
uv run trade-ledger import-notes --dir DIR --account NAME
uv run trade-ledger prices --refresh
uv run trade-ledger tax-year YEAR [--mode live|paper|demo|all]
uv run trade-ledger report [--week YYYY-MM-DD] [--mode live|paper|demo|all]

uv run trade-bot --bot SLUG [--dry-run] [--once]
uv run trade-bot new NAME
```

12. A weekly routine

- Before every order: plan the trade in the Journal. No plan, no order.
- Daily, two minutes: Dashboard for stale bots and unacknowledged alerts; open trades still have their stops.
- Weekly, thirty minutes: Sync or import; Refresh prices; close and review the week's trades; read the stage gate; generate the weekly report; note one lesson.
- Monthly: Steuer year view for the Freigrenze meters; export the venue CSVs and keep them (foreign venues: you must keep your own records).
- Never: type a key into a website, share an account, or run a bot with a withdrawal-capable key.

13. Troubleshooting

Symptom	Cause and fix
Values show "—"	No cached price or FX rate. Portfolio → Refresh prices. Check the instrument has a price source and symbol (Settings → Instruments).
Transactions with pending EUR value	Non-EUR rows waiting for ECB rates. Refresh prices or run a sync; both resolve them.
OKX sync warns about three months	The API window. Import the quarterly archive CSV once; the API keeps it current afterwards.
Sync run says missing credentials	The account's credential prefix does not match the keys in <code>.env</code> , or the server was not restarted after editing <code>.env</code> .
Bot is stale	No heartbeat by the deadline. Check the run log (Runs tab), the env file, and that the server (local) or timer (remote) is running.
Bot finishes every run as skipped	No preset assigned, or the bot is disabled. Assign a version on the Config tab.

K4 reconciliation mismatch	The exchange holds something the journal does not know (or vice versa). Use a sub-account per bot; square the journal, then Resume.
K1 shows a warning, not a value	The market is not EUR-quoted, so no equity in EUR is reported. Use an EUR-quoted account for K1 coverage.
Telegram test fails	Wrong token or chat id, or the bot has not been started in the chat. Send it a message first, then test.
Frontend shows 'not built'	Run <code>cd frontend && npm ci && npm run build</code> , then restart the server.

Repository: <https://github.com/AmirrezaRoodsaz/trade-ledger> · MIT license · Tax module: Berechnung — mit Steuerberater prüfen.